

STYLE360

Next-Generation AI-Driven Virtual Try-On, 3D Mesh Reconstruction, and Real-Time WebGL Fashion Fitting Studio

PROJECT TITLE

Style360 AI Virtual Studio

SYSTEM ARCHITECTURE

Microservice & Cloud AI Pipeline

CORE TECHNOLOGIES

PHP 8.2, MySQL PDO, Three.js
WebGL

AI & CLOUD SERVICES

Fal.ai VTON, Tripo3D V3, Gemini
Vision, R2

TARGET EXAMINATION

Final Year Academic Project Viva

DOCUMENTATION SCOPE

Complete File-by-File &
Architectural Audit

1.1 Abstract & Core Vision

Style360 is an innovative, production-grade web platform engineered to eliminate one of e-commerce's most costly bottlenecks: clothing return rates caused by inaccurate fit perception and static 2D product photography. The platform bridges computer vision, generative artificial intelligence, and interactive WebGL 3D computer graphics into a unified browser-based experience.

Unlike conventional virtual try-on systems that merely superimpose flat 2D clothing textures onto arbitrary images (often resulting in unnatural warping, gender mismatches, and severe body part cropping), Style360 features a **dual-stage AI intelligence pipeline** coupled with an interactive **360-degree 3D avatar viewport**.

1.2 The Core Problem Statement

Traditional E-Commerce	The Fundamental Defect	How Style360 Solves It
Static 2D Images	Users cannot evaluate how garments drape around the torso, waist, or back profile.	Trip3D V3 360° Mesh: Generates an interactive 80,000-face textured 3D avatar viewable from any orbital perspective.
High Return Rates (30-40%)	Customers order multiple sizes and return ill-fitting or unflattering outfits.	Photorealistic FASHN v1.6 VTON: Calculates precise fabric draping, shadow synthesis, and realistic body contact.
Gender / Category Mismatches	Users upload male photos for female dresses, producing grotesque AI glitches.	Gemini Vision AI Guard: Automated pre-flight validation preventing cross-gender try-ons in ~1.2 seconds.
Body Cropping & Distortion	Standard diffusion models crop full-body photos at the knees or warp head scales.	Intelligent Pillarbox Resampling: Custom GD algorithm preserving 100% vertical stature and true head-to-body proportions.

Key Viva Takeaway: When examiners ask "What makes Style360 different from an ordinary photo editor or mobile try-on app?", emphasize: **We combine photorealistic 2D generative diffusion with single-view 3D mesh reconstruction and zero-overhead WebGL rendering directly in the client browser.**

Style360 is built using a modern **hybrid decoupled client-server architecture**. The frontend operates as a lightweight, zero-compilation Single Page Interface (SPI) communicating via asynchronous JSON REST APIs with a PHP 8.2 backend gateway. The backend acts as a resilient orchestrator and security proxy connecting to three cutting-edge external cloud AI engines and Cloudflare R2 object storage.

2.1 High-Level Architecture Diagram

1. Client Presentation Tier (Browser / Three.js)

Vanilla JS + WebGL

Responsive UI, Drag-and-Drop Image Uploader, Garment Catalog with Category/Gender sync, Three.js 360° OrbitControls Viewport, High-DPI Canvas Rendering, AI Stylist Chat Widget.

↓ **Asynchronous JSON REST Requests (multipart/form-data & JSON)** ↓

2. Backend Application & Orchestration Gateway

PHP 8.2 + PDO MySQL

Input sanitization, GD in-memory thumbnail resampling, MD5 caching engine, Session/Auth controller, CORS proxy for 3D binary assets ([proxy_glb.php](#)), self-healing database schema synchronizer.

↓ **Parallel REST / Webhook Integration Pipelines** ↓

3. Intelligent AI & Cloud Services Tier

External Cloud Microservices

Google Gemini 1.5/2.0	Multimodal Vision AI: Pre-flight gender verification, garment category detection, and conversational fashion advice.
Fal.ai FASHN v1.6	Diffusion-based latent virtual try-on engine for high-resolution photorealistic garment replacement and drape synthesis.
Tripo3D V3 API	Single-view image-to-3D mesh reconstruction pipeline producing 80,000-face GLB models with PBR textures in ~12 seconds.
Cloudflare R2	S3-compatible global object storage with zero egress fees for permanent storage of high-res try-on renders and 3D GLB assets.

2.2 Step-by-Step User Execution Pipeline

- 1. Model Upload & Pre-Flight Validation:** User uploads a full-body portrait. Frontend sends image thumbnail to [backend/validate_gender.php](#). Gemini Vision AI inspects biological features vs selected catalog garment. If a mismatch is detected (e.g. male user selecting female evening gown), UI automatically flips or alerts the user with graceful explanation.

2. **Aspect-Ratio Standardization:** `backend/FalService.php` calculates original aspect ratio. If full-body (ratio ≤ 0.68), it applies background-sampled pillarboxing onto a 576x1024 / 768x1024 canvas, ensuring zero limb truncation.
3. **2D Photorealistic Try-On Synthesis:** Request is dispatched to `fa1-ai/fashn/tryon/v1.6`. The model warps the garment according to user posture, synthesizes natural fabric tension, cast shadows, and outputs a photorealistic try-on image (~8-12 seconds).
4. **Automated Background Stripping:** `FalService::removeBackground()` strips noisy studio walls/floors, providing a clean transparent subject to the 3D pipeline.
5. **Tripo3D V3 80k Mesh Reconstruction:** `backend/generate_3d.php` packages the clean subject into an optimized subdivision geometry payload (80,000 polygon limit, PBR enabled) and polls the Tripo task until the `.glb` mesh is compiled.
6. **Three.js Real-Time Orbit Rendering:** Mesh is streamed through `backend/proxy_glb.php`. Three.js calculates bounding box (`THREE.Box3`), scales avatar to exactly 1.75m human height, anchors feet at $y=0$, applies 16x anisotropic filtering to all textures, and positions camera to fill 80% of canvas height.

Technology Stack & Rationale

Examiners frequently ask: "Why did you select this specific technology stack instead of alternative frameworks?" Below are the academic and technical justifications for every architectural decision made in Style360.

Technology	Role in Style360	Technical Rationale	Why Not the Alternative?
PHP 8.2+	Backend API Gateway & Business Logic	Stateless execution model, zero daemon overhead, native cURL binary stream handling, battle-tested PDO database pooling, and ultra-fast cold start times.	vs Node.js/Python: Node single-thread event loops can freeze under heavy image processing; Python web servers (Flask/Django) require complex WSGI/Gunicorn daemon management.
Vanilla JS (ES6+)	Frontend Client Architecture	Zero build step, instant browser compilation, microsecond DOM updates, and direct uninhibited WebGL memory management without virtual DOM overhead.	vs React / Next.js: React adds 150KB+ bundle weight, and React's virtual DOM reconciliation loop creates garbage collection pauses during 60 FPS Three.js rendering.
Three.js (r128)	Client-Side 3D Graphics Engine	Industry standard WebGL abstraction layer. Provides GLTFLoader, OrbitControls, PCF soft shadow maps, PBR material shaders, and cross-platform hardware acceleration.	vs Babylon.js / Unity WebGL: Unity WebGL builds exceed 30MB+ download sizes; Three.js is lightweight (<600KB) and operates natively within standard HTML5 canvases.
MySQL 8.0 with PDO	Relational Database Management	ACID compliance, relational integrity with foreign keys, prepared statements protecting against SQL injection, and structured indexing for user history queries.	vs MongoDB: MongoDB lacks strict schema enforcement for transaction-critical user orders and financial custom requests; MySQL guarantees consistency.
Fal.ai FASHN v1.6	Latent Diffusion 2D Try-On	Specialized commercial latent diffusion model trained specifically on fashion apparel. Handles complex garment classes (tops, bottoms, full-body dresses) with drape physics.	vs IDM-VTON / Open-Source: IDM-VTON requires high-end dedicated A100 GPU hosting (\$2.50/hr), takes 40+ seconds per try-on, and frequently fails on complex dresses.

Technology	Role in Style360	Technical Rationale	Why Not the Alternative?
Tripo3D V3 API	Single-View 3D Reconstruction	Direct image-to-mesh reconstruction generating watertight 80k-face GLB models with diffuse and normal maps in 10-15 seconds.	vs Rodin / SF3D / Trellis: Trellis and Rodin take 90-180 seconds to compile NeRF/Gaussian splats and require gigabytes of VRAM. Tripo V3 delivers instant commercial latency.
Google Gemini Vision AI	Multimodal Intelligence & Safety	Fast multimodal reasoning. Classifies gender, clothing category, and inappropriate content with structured JSON responses in ~1.2 seconds.	vs Local OpenCV / ResNet: Classical classifiers struggle with diverse lighting, complex postures, and ethnic fashion styles (sarees, lehengas, sherwanis).
Cloudflare R2	Object Storage CDN	100% S3 API compatibility with zero egress bandwidth fees. Extremely cost-effective for serving large 3D GLB binary files globally.	vs AWS S3: AWS S3 charges exorbitant egress data transfer fees (\$0.09/GB), which rapidly bankrupts applications serving 5-15MB 3D models to thousands of users.

Comprehensive File-by-File Code Walkthrough

This section breaks down **every critical file in the Style360 codebase**. In your viva, examiners frequently ask: "Open file X and explain line Y" or "Where does the database connection happen?". Use this reference to answer with absolute confidence.

4.1 Backend Engine & Core Services (/backend)

backend/config.php

Configuration Manager

Purpose & Mechanics: Central configuration hub of the entire application. Loads environment variables from the root `.env` file into PHP `$_ENV` and `getenv()`.

- Defines critical global constants: `FAL_KEY`, `TRIPO3D_API_KEY`, `GEMINI_API_KEY`, `MOCK_VTON`.
- Defines Modal.com pipeline endpoints: `MODAL_PIPELINE_URL`, `MODAL_SF3D_URL`, `MODAL_TRELLIS_URL`.
- Defines Cloudflare R2 credentials: `CLOUDFLARE_R2_ACCOUNT_ID`, `CLOUDFLARE_R2_ACCESS_KEY_ID`, `CLOUDFLARE_R2_BUCKET_NAME`.
- Suppresses runtime error display in production (`ini_set("display_errors", "0")`) while logging clean API errors.

backend/database.php

Self-Healing PDO Database Connection

Purpose & Mechanics: Robust PDO connection manager with automated self-healing and schema migration capabilities.

- **Multi-Host Fallback Loop:** Iterates across `DB_HOST`, `127.0.0.1`, `localhost`, and port overrides to guarantee successful database handshake regardless of local XAMPP/Docker configuration.
- **Auto-Creation:** If database `style360_v2` does not exist, connects to MySQL root and executes `CREATE DATABASE IF NOT EXISTS style360_v2` dynamically.
- **Auto-Migration:** Automatically creates missing tables (`custom_requests`, `notifications`, `try_on_history`) on first boot.
- **Schema Evolution:** Detects if `result_3d_url` column is `VARCHAR(255)` and upgrades it to `TEXT` to prevent truncation of long signed AWS/CloudFront tokens (>900 chars).
- **Mock Data Purge:** Automatically sweeps and purges legacy placeholder demo images (e.g. `images/cat_wedding_men.png`) to maintain pristine user data.

backend/FalService.php

Core Virtual Try-On Engine

Purpose & Mechanics: Over 1,000 lines of specialized image processing and API integration logic for Fal.ai services.

- `standardizeModelImage()` : Reads image metadata and applies smart letterboxing/pillarboxing. If a full-body portrait (9:16 or 2:3) is provided, it scales the height to fill 1024px, samples the background corner pixel color (e.g. `imagecolorat($srcImg, 4, 4)`), and pads the sides. This ensures **100% of the person's feet and head are preserved** without truncation.
- `virtualTryOn()` : Dispatches payload to `https://fal.run/fal-ai/fashn/tryon/v1.6`. Configures category mapping (`tops` , `bottoms` , `one-pieces`), garment warp settings, and head-to-body proportion constraints.
- `removeBackground()` : Calls Fal.ai BiRefNet background removal model to isolate the subject from studio furniture before 3D reconstruction.
- `recolorGarment()` : Executes inpainting via FLUX General Inpainting to alter garment hues based on hex codes.

backend/tryon.php

Main Try-On API Endpoint (/api/try-on)

Purpose & Mechanics: Primary POST controller receiving virtual try-on requests from the frontend studio.

- Validates request parameters (`person_image` , `garment_image` , `category` , `aspect_ratio`).
- Detects full-body stature based on height-to-width ratios (≤ 0.68) and dynamically configures 9:16 portrait target dimensions (576x1024).
- Executes Fal.ai VTON pipeline, falls back to Modal.com if configured, and supports instant 2-second realistic mock simulation mode when testing UI offline.
- Saves successful generation records directly into MySQL `try_on_history` and returns the new `history_id` to the client.

backend/generate_3d.php

Tripo3D V3 High-Detail 3D Generator

Purpose & Mechanics: Handles the 2D-to-3D mesh reconstruction lifecycle.

- **Credit Conservation & Hash Caching:** Computes the MD5 checksum of the input try-on image. If the exact model has already been reconstructed in `user_3d_models`, returns the cached `.glb` URL immediately, saving API credits and eliminating redundant wait times.
- **Automated Background Removal:** Calls `FalService::removeBackground()` to eliminate noisy walls and floors, preventing blocky mesh artifacts around the avatar.
- **Optimized V3 Payload:** Sends high-detail subdivision geometry configuration (`face_limit: 80000` , `texture: true` , `pbr: true`) directly via base64, skipping slow image upscalers to maintain an ultra-fast ~10-12 second response time.
- **Polling Engine:** Asynchronously polls Tripo3D task status until the GLB URL is generated, then updates the database record.

backend/proxy_glb.php

CORS & Local Binary Streaming Proxy

Purpose & Mechanics: Solves Cross-Origin Resource Sharing (CORS) blocks when Three.js loads 3D models from third-party CDNs (Tripo3D CloudFront or Cloudflare R2).

- Streams binary GLB data with proper HTTP headers: `Content-Type: model/gltf-binary` , `Access-Control-Allow-Origin: *` .
- Supports local disk caching in `temp/glb_cache/` so repeat 3D views load in under 10 milliseconds without re-fetching from the cloud.
- Supports model resolution by URL (`?url=...`) or by database ID (`?id=...`).

backend/validate_gender.php

Gemini Vision AI Gender Guard

Purpose & Mechanics: Pre-flight safety check preventing cross-gender try-on mismatches.

- Two-Tier Performance:** Tier 1 uses ultra-fast regex inspection on the uploaded filename (detects keywords like `men` , `male` , `dress` , `gown`) in <1ms. Tier 2 uses Google Gemini Vision AI.
- In-Memory GD Downsampling:** Downsamples the user image in memory to a max 320px JPEG thumbnail (~10KB) before sending to Gemini REST API. This reduces latency from ~7 seconds down to ~1.2 seconds.
- Returns structured JSON (`match: true/false` , `detected_gender` , `confidence`). If a mismatch occurs, frontend smoothly auto-switches gender or prompts user.

backend/verify_garment.php

Garment AI Safety & Category Verifier

Purpose & Mechanics: Validates user-uploaded custom clothing images before try-on processing.

- Uses Gemini Vision AI to confirm that the uploaded file is indeed clothing apparel (and not a random object, pet, or NSFW image).
- Detects garment sub-type (`top` , `bottom` , `dress` , `outerwear`) and recommended drape settings.

backend/auth.php, check_session.php, logout.php

Authentication & Session Management

Purpose & Mechanics: Secure user credential handling using modern cryptographic standards.

- Passwords hashed using PHP's native `password_hash($pass, PASSWORD_BCRYPT)` with default cost factor 10.
- Authentication tokens managed via native PHP sessions with cookie safety flags (`HttpOnly` , `SameSite=Lax`).
- `check_session.php` returns current authenticated user ID, name, email, and role (admin/user) to protect private routes.

backend/history.php & get_history.php

Try-On Archive Management

Purpose & Mechanics: Handles retrieval, pagination, and deletion of user virtual try-on creations.

- Joins `try_on_history` with `garment` table to retrieve outfit titles, categories, and timestamps.
- Provides one-click deletion endpoint that cleans database rows and orphaned local cached files.

backend/custom_requests.php & admin-requests.php

Custom Tailoring Workflow & Admin Portal

Purpose & Mechanics: Complete bespoke fashion tailoring inquiry workflow.

- Users can submit custom garment modification requests (custom top color, bottom color, fit notes, reference images).
- Stored with state machine tracking: `Pending Review` → `In Progress` → `Ready for Download` .
- `admin-requests.php` provides administrative moderation dashboard to approve, reject, or upload tailored renders.

backend/chat.php & chat_multimodal.php

Gemini Conversational AI Stylist

Purpose & Mechanics: Conversational fashion advisory engine powered by Google Gemini.

- Maintains context-aware fashion advice for body types, skin tones, wedding themes, and casual occasions.
- `chat_multimodal.php` accepts user photo + style query and returns personalized garment recommendations from the catalog.

backend/router.php

Built-in Development Server Routing

Purpose & Mechanics: Enables zero-dependency local development via PHP's built-in CLI server (`php -S localhost:8000 router.php`).

- Routes API endpoints (e.g. `/api/try-on`) to corresponding PHP scripts.
- Serves static assets (HTML, CSS, JS, GLB, images) with correct MIME types without requiring Apache or Nginx.

4.2 Frontend Architecture & WebGL Graphics (/frontend)

frontend/js/viewer3d.js (Style360Viewer)

Three.js 3D WebGL Viewport Controller

Purpose & Mechanics: Over 1,200 lines of high-performance WebGL graphics rendering logic.

- **High-DPI Antialiased Renderer:** Configures `THREE.WebGLRenderer` with `antialias: true`, `powerPreference: "high-performance"`, and `renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2))` for crystal-clear retina rendering.
- **Anisotropic Texture Filtering:** Traverses all loaded GLB mesh materials (diffuse `map`, `normalMap`, `roughnessMap`) and activates **16x Anisotropic Filtering** (`tex.anisotropy = renderer.capabilities.getMaxAnisotropy()`) with `LinearMipmapLinearFilter`. This eliminates texture blurriness during 360° orbital rotation.
- **Dynamic Box3 Auto-Fit & Camera Framing:** Computes the mesh bounding box (`new THREE.Box3().setFromObject(model)`), automatically scales the model to a realistic 1.75m height, centers X/Z at (0, 0), and aligns feet at $y=0$ on the studio podium. Dynamically calculates optimal camera distance using the vertical field of view equation:
$$\text{optimalDist} = (\text{actualHeight} / (2 * \text{Math.tan}(\text{vFov} / 2))) / 0.80$$

This guarantees the avatar occupies exactly **80% of the viewport height** regardless of screen resolution.
- **Smooth Shading & Vertex Normal Recomputation:** Executes `child.geometry.computeVertexNormals()` and sets `mat.side = THREE.DoubleSide`, preventing black polygons or facet lines.
- **Studio Lighting Rig:** Soft ambient light (0.95 intensity), primary directional key light with 2048x2048 PCF soft shadows, rim light for silhouette separation, and subtle studio fog.

frontend/js/home.js

Main Application State Orchestrator

Purpose & Mechanics: Coordinates global UI state, catalog rendering, try-on stage transitions, and zero-flicker routing.

- Manages central state object: `tryonState = { userPhotoData, selectedGarment, selectedGender, currentCollectionFilter, historyId }`.
- **Zero-Flicker Instant Route Preloader:** Synchronously parses `window.location.hash` (`#tryon`, `#category-section`) and responds to dynamic `hashchange` events, completely eliminating landing page flash when returning from external galleries.
- Renders catalog garment cards with category chips (Wedding, Formal, Casual, Party) and gender toggle filters.
- Synchronizes model gender with garment category: selecting a men's tuxedo automatically configures male avatar defaults.
- Exposes global bridge `window.selectGarmentById(garment, optionalUserPhoto)` allowing external modules and the AI Stylist chatbot to seamlessly transfer selected garments and user portraits directly into active Try-On studio slots.
- Handles dual-stage viewport switching between 2D try-on comparison mode and 3D WebGL orbit mode.

frontend/js/studio.js

Try-On Studio Controller

Purpose & Mechanics: Drives user interaction inside the Try-On Studio.

- Provides drag-and-drop file uploaders with instant client-side image preview and Base64 encoding.
- Executes pre-flight validation against `backend/validate_gender.php` before submitting expensive try-on jobs.
- Displays animated step-by-step progress loaders ("*Analyzing Pose* → *Warping Fabric* → *Rendering Shadows*").
- Controls bespoke custom outfit color request modal with native dual color pickers, quick-swatches, and instant tailoring dispatch.

frontend/js/models3d.js

3D Avatar Gallery & Interactive Modal

Purpose & Mechanics: Dedicated controller for `frontend/my-3d-models.html`.

- Fetches user's generated 3D models from `backend/get_3d_models.php`.
- Renders responsive model cards showing 2D preview thumbnails, creation dates, and polygon counts.
- Configures Google `<model-viewer>` with calibrated PBR lighting (`exposure="0.92"` , neutral studio HDRI) preventing blown-out highlights on 3D avatar meshes.
- Provides the "🔍 **View Model**" action button which opens an expanded interactive 3D modal with OrbitControls and full-screen inspection mode.

frontend/js/chatbot.js

Multimodal Gemini AI Stylist & Try-On Link

Purpose & Mechanics: Floating conversational and multimodal AI stylist integrated into the bottom-right of the screen.

- **Multimodal Visual Stylist:** Allows users to upload their own photo directly into the chat and ask "*What outfit suits me?*" or "*Suggest wedding clothes for this photo*".
- Connects to `backend/chat_multimodal.php` and Google Gemini 1.5/2.0 Multimodal Vision to analyze skin tone, facial features, gender, and occasion.
- Queries the internal MySQL database for available stock matching the user, rendering interactive recommendation cards inside the chat bubble.
- Clicking "🔥 **Try On in Studio**" directly auto-loads the user's uploaded portrait and the selected outfit into the Try-On studio and scrolls to the workspace.

Style360 uses a normalized relational database schema (`style360_v2`) designed for referential integrity, high query throughput, and ACID compliance.

5.1 Entity-Relationship (ER) Schema Overview

Table Name	Key Columns & Data Types	Indexes & Foreign Keys	Purpose in System
<code>users</code>	<code>id INT AUTO_INCREMENT (PK)</code> <code>name VARCHAR(100)</code> <code>email VARCHAR(150) UNIQUE</code> <code>password VARCHAR(255)</code> <code>role ENUM('user','admin')</code> <code>created_at DATETIME</code>	PK on <code>id</code> UNIQUE on <code>email</code>	Stores user profiles, roles, and bcrypt-hashed security credentials.
<code>garments</code>	<code>id INT AUTO_INCREMENT (PK)</code> <code>title VARCHAR(200)</code> <code>category VARCHAR(50)</code> <code>gender ENUM('male','female','unisex')</code> <code>image_url VARCHAR(500)</code> <code>model_3d_url TEXT NULL</code> <code>price DECIMAL(10,2)</code>	PK on <code>id</code> INDEX on <code>category</code> INDEX on <code>gender</code>	Product catalog items available for virtual try-on and 3D preview.
<code>try_on_history</code>	<code>id INT AUTO_INCREMENT (PK)</code> <code>user_id INT (FK nullable)</code> <code>garment_id INT (FK nullable)</code> <code>result_image_url VARCHAR(500)</code> <code>result_2d_url VARCHAR(500)</code> <code>result_3d_url TEXT NULL</code> <code>created_at DATETIME</code>	PK on <code>id</code> FK to <code>users(id)</code> FK to <code>garments(id)</code> INDEX on <code>created_at</code>	Permanent log of all generated 2D try-on renders and compiled 3D GLB assets.
<code>user_3d_models</code>	<code>id INT AUTO_INCREMENT (PK)</code> <code>tryon_history_id INT</code> <code>task_id VARCHAR(100)</code> <code>model_url TEXT</code> <code>preview_image_url VARCHAR(500)</code> <code>outfit_title VARCHAR(255)</code> <code>created_at DATETIME</code>	PK on <code>id</code> INDEX on <code>tryon_history_id</code> INDEX on <code>task_id</code>	Dedicated 3D avatar registry for quick gallery population and hash cache matching.
<code>custom_requests</code>	<code>id INT AUTO_INCREMENT (PK)</code> <code>user_id INT (FK)</code>	PK on <code>id</code> INDEX on <code>status</code>	Bespoke tailoring request workflow

Table Name	Key Columns & Data Types	Indexes & Foreign Keys	Purpose in System
	outfit_id INT (FK) top_color VARCHAR(100) bottom_color VARCHAR(100) notes TEXT result_image_url VARCHAR(500) status VARCHAR(50) created_at DATETIME	FK to users(id)	tracking from initial submission to completion.
notifications	id INT AUTO_INCREMENT (PK) user_id INT (FK) title VARCHAR(255) message TEXT is_read TINYINT(1) DEFAULT 0 created_at DATETIME	PK on id INDEX on (user_id, is_read)	User alert system notifying when 3D models or custom tailoring requests are ready.

Key Engineering Challenges & Solutions

Academic examiners value the candidate's problem-solving acumen. Below are the five most significant technical roadblocks encountered during the engineering of Style360 and how they were overcome.

Challenge 1: CORS Header Absence on Remote 3D GLB Endpoints

The Problem: When Three.js `GLTFLoader` loaded binary `.glb` files generated by Tripo3D or hosted on external CDNs, modern web browsers blocked the download due to strict Cross-Origin Resource Sharing (CORS) security policies.

The Solution: Implemented a high-performance streaming CORS proxy in `backend/proxy_glb.php`. When Three.js requests a model, it requests `backend/proxy_glb.php?url=<encoded_url>`. The PHP script performs server-to-server streaming with `Content-Type: model/gltf-binary` and `Access-Control-Allow-Origin: *`. Furthermore, it caches the binary file locally in `temp/glb_cache/`, speeding up subsequent model loads by 95%.

Challenge 2: Inconsistent 3D Model Scales & Centering

The Problem: AI-reconstructed 3D meshes arrived with unpredictable coordinate spaces: some models were tiny (0.1 units), others enormous (50 units), some floating in air, and others oriented with Z-axis pointing up instead of Y-axis.

The Solution: Engineered a robust 8-step normalization algorithm in `frontend/js/viewer3d.js`:

```
// 1. Compute raw bounding box & detect Z-up orientation
var rawBox = new THREE.Box3().setFromObject(model);
var rawSize = new THREE.Vector3();
rawBox.getSize(rawSize);
if (rawSize.z > rawSize.y * 1.5) { model.rotation.x = -Math.PI / 2; }

// 2. Uniform scale to exact 1.75m human height
var sf = 1.75 / (rawSize.y > 0.05 ? rawSize.y : 1.0);
model.scale.set(sf, sf, sf);
model.updateMatrixWorld(true);

// 3. Grounding: Center X and Z at 0, place lowest Y vertex at y=0 (podium floor)
var grounded = new THREE.Box3().setFromObject(model);
var gc = new THREE.Vector3();
grounded.getCenter(gc);
model.position.x -= gc.x;
model.position.z -= gc.z;
model.position.y -= grounded.min.y;
```

Challenge 3: Texture Blurriness & Pixelation During Orbit Rotation

The Problem: When users rotated the 3D model, fabrics appeared blurry, pixelated, and jagged due to default bilinear texture sampling and standard canvas pixel scaling.

The Solution: Applied an advanced graphics pipeline in Three.js:

- Activated **High-DPI Retina resolution:** `renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2))`.
- Enabled **16x Anisotropic Texture Filtering:** `tex.anisotropy = renderer.capabilities.getMaxAnisotropy()` on all material maps.
- Configured **Trilinear Mipmapping:** `tex.minFilter = THREE.LinearMipmapLinearFilter` and `tex.magFilter = THREE.LinearFilter`.
- Enforced **sRGB Color Space:** `renderer.outputColorSpace = THREE.SRGBColorSpace` to ensure color accuracy without dark banding.

Challenge 4: Lower-Body Truncation on Full-Body Photos

The Problem: Default diffusion try-on APIs forced square or 3:4 crops, chopping off the legs and shoes of standing users.

The Solution: Built intelligent aspect-ratio detection in `backend/FalService.php`. If the input image has a tall portrait aspect ratio (≤ 0.68), the system preserves 100% of vertical height, samples the background corner pixel color, and creates a seamless 576x1024 canvas with proportional pillarboxing. Full-body models remain intact from head to toe.

Top 21 Viva Questions & Model Answers

These 21 questions represent the exact lines of inquiry examiners pursue during final project examinations. Study both the **Direct Answer** and the **Examiner Insight**.

Q1 What is the primary novelty and technical contribution of Style360?

Model Answer: Style360 bridges 2D generative diffusion with single-view 3D mesh reconstruction. Most platforms offer either flat 2D image overlays (which lack depth) or static pre-rigged 3D avatars (which lack real user likeness). Style360 takes a single photo of any user, applies photorealistic garment warping and shadow synthesis via Fal.ai FASHN v1.6, and converts that 2D result into an 80,000-polygon interactive 3D WebGL avatar within ~12 seconds.

Examiner Psychology: Examiners want to ensure this isn't just a simple UI wrapper around a single API. Highlight the orchestration pipeline.

Q2 Why did you use PHP for the backend instead of Node.js or Python?

Model Answer: PHP 8.2 provides an isolated, stateless execution environment where each HTTP request runs in its own process. This eliminates shared-memory leaks and prevents single-thread event loop blocking caused by heavy base64 image decoding. Furthermore, PHP's native cURL handles binary streaming directly to the client without buffering multi-megabyte GLB models into RAM, offering superior memory efficiency.

Examiner Psychology: Don't apologize for using PHP; justify it using systems engineering principles (statelessness, memory isolation, binary streaming).

Q3 How does Style360 solve the CORS problem when loading 3D models?

Model Answer: When Three.js loads GLB assets from external CDNs (like Tripo3D's CloudFront), browsers block the request if the CDN lacks the `Access-Control-Allow-Origin` header. We solved this by developing `backend/proxy_glb.php`, which streams binary data through our origin server with full CORS headers (`Access-Control-Allow-Origin: *`, `Content-Type: model/gltf-binary`) while caching models on local disk for instant repeat rendering.

Examiner Psychology: Proves you understand real-world browser security models and HTTP protocol nuances.

Q4 Explain the mathematical calculation used to frame the 3D model in Three.js.

Model Answer: We use the vertical field-of-view trigonometry formula. First, `THREE.Box3` computes the model's bounding box height (H). Given camera vertical FOV θ in radians: $\text{distance} = \frac{H}{2 \cdot \tan(\theta / 2) \cdot \text{targetCoverage}}$. We set $\text{targetCoverage} = 0.80$, ensuring the avatar consistently occupies exactly 80% of the canvas height, regardless of screen aspect ratio or window resizing.

Examiner Psychology: Examiners love trigonometry and 3D projection mathematics. This demonstrates deep computer graphics competency.

Q5 How do you prevent cross-gender mismatches during virtual try-on?

Model Answer: We implemented a dual-stage validation guard in `backend/validate_gender.php`. Tier 1 performs regex checks on the uploaded filename in <1ms. Tier 2 resamples the image into an in-memory 320px thumbnail and queries Google Gemini Vision AI. If a mismatch is detected (e.g., male model with female evening dress), the system automatically syncs the UI catalog filter or alerts the user before burning expensive try-on compute credits.

Examiner Psychology: Demonstrates practical safety engineering and cost optimization in AI systems.

Q6 What is Anisotropic Filtering and why was it necessary in Three.js?

Model Answer: Standard isotropic mipmapping samples textures as symmetrical squares. When a 3D model is viewed at oblique angles (such as rotating an avatar's shoulders or waist), textures become severely blurred. Anisotropic filtering samples textures along non-square projections. We query hardware capabilities using `renderer.capabilities.getMaxAnisotropy()` (up to 16x) to ensure fabric weaves and patterns remain razor-sharp under rotation.

Examiner Psychology: Demonstrates mastery of GPU rasterization and WebGL rendering pipelines.

Q7 How does the system handle database resiliency if tables are missing?

Model Answer: In `backend/database.php`, we implemented self-healing connection logic. If database `style360_v2` does not exist, the script connects to MySQL root and creates it automatically. It then executes conditional `CREATE TABLE IF NOT EXISTS` statements and dynamically alters columns (such as expanding `result_3d_url` from `VARCHAR` to `TEXT` to accommodate 900+ character signed CDN URLs).

Examiner Psychology: Shows enterprise-grade software architecture principles and zero-maintenance deployment design.

Q8 What is the role of Cloudflare R2 in this architecture?

Model Answer: Cloudflare R2 is an S3-compatible object storage system. 3D GLB models and high-res try-on renders average 5 to 15 megabytes each. Standard cloud providers (like AWS S3) charge heavy egress bandwidth fees (\$0.09 per GB transferred). Cloudflare R2 has **zero egress bandwidth fees**, making it infinitely more cost-effective for serving large 3D graphic assets globally.

Examiner Psychology: Demonstrates cloud financial management (FinOps) and pragmatic infrastructure selection.

Q9 How do you prevent API credit exhaustion during testing or repeat requests?

Model Answer: In `backend/generate_3d.php`, we compute an MD5 hash checksum of the input image data (`$imageHash = md5($cleanBase64)`). Before contacting Tripo3D, we query the `user_3d_models` table. If the same image was previously reconstructed, we return the cached GLB URL instantly in <50ms, preserving API credits and providing instantaneous response times.

Examiner Psychology: Caching strategies show that you design for economic scalability and efficiency.

Q10 How do you ensure user passwords are secure in the database?

Model Answer: Passwords are never stored in plaintext. We utilize PHP's native `password_hash($password, PASSWORD_BCRYPT)` with an adaptive work factor cost of 10. Bcrypt includes a cryptographically secure random salt per password, rendering rainbow table attacks completely ineffective. Verification uses timing-attack-safe `password_verify()`.

Examiner Psychology: Security is a mandatory viva question. Mentioning bcrypt and salts shows solid fundamentals.

Q11 What is the difference between GLTF and GLB formats?

Model Answer: GLTF (Graphics Language Transmission Format) is the JSON-based standard for 3D scenes, but it often references external `.bin` files for geometry and separate image files for textures. **GLB is the binary container format of GLTF:** it packages the JSON scene description, binary vertex/index buffers, and embedded PBR texture images into a single self-contained binary file, which simplifies network transmission and streaming.

Examiner Psychology: Proves understanding of 3D asset pipeline formats.

Q12 How does the Gemini AI Stylist maintain conversational context?

Model Answer: In `frontend/js/chatbot.js`, client-side conversation history is aggregated into a structured chat payload. When calling `backend/chat.php`, previous user and model turns are packaged and injected into the Gemini 1.5 system instruction, allowing the model to recall previous user preferences (e.g. skin tone, body type, preferred color palette) across multi-turn dialogues.

Examiner Psychology: Tests knowledge of LLM context management and multi-turn prompt engineering.

Q13 Why did you downsample images in memory before calling Gemini Vision AI?

Model Answer: User smartphone cameras produce 12-48 megapixel photos (4-15MB each). Sending multi-megabyte payloads to external vision APIs introduces 5 to 7 seconds of network upload latency. In `backend/validate_gender.php`, we use PHP GD to create an in-memory 320px JPEG thumbnail (~12KB). Gemini only needs coarse visual features to classify gender and garment type, cutting round-trip latency to ~1.2 seconds.

Examiner Psychology: Shows deep appreciation for real-world performance optimization and latency reduction.

Q14 How are database SQL injection vulnerabilities prevented?

Model Answer: 100% of database interactions in Style360 utilize PDO (PHP Data Objects) with parameterized prepared statements (e.g. `$stmt->prepare("SELECT * FROM users WHERE email = :email")` and `$stmt->execute([':email' => $email])`). User inputs are sent separately from the SQL command structure, ensuring malicious query fragments are treated strictly as data literals.

Examiner Psychology: A classic security benchmark question. PDO prepared statements is the gold-standard answer.

Q15 How does the 3D viewer handle models with Z-up coordinates?

Model Answer: Some 3D modeling packages (such as Blender or CAD tools) treat the Z-axis as vertical height ("Z-up"), while Three.js and OpenGL treat the Y-axis as height ("Y-up"). In `viewer3d.js`, we compare the bounding box dimensions: if `rawSize.z > rawSize.y * 1.5`, the model is clearly laying down or exported in Z-up mode. We programmatically apply a rotation of `model.rotation.x = -Math.PI / 2` to instantly correct its orientation.

Examiner Psychology: Proves hands-on debugging experience with real 3D geometry assets.

Q16 What happens if the internet connection or an external AI API fails?

Model Answer: Style360 is built with defensive fault tolerance: 1. The backend implements cURL timeout guards (e.g. 30s connect timeout). 2. A `MOCK_VTON` configuration flag allows the system to seamlessly switch to simulated local try-on responses for offline testing or presentation environments. 3. Frontend error handlers intercept HTTP 502/504 errors and present friendly, actionable UI toasts instead of crashing the interface.

Examiner Psychology: Examiners love to ask "What if your API goes down during our grading?" Having a mock simulation mode is a huge positive mark.

Q17 Explain how the custom tailoring order workflow operates.

Model Answer: Managed via `custom_requests.php` and `admin-requests.php`. When a user customizes an outfit (e.g., changes top color to navy and bottom to charcoal), a record is inserted into `custom_requests` with status `'Pending Review'`. The administrator reviews the request in the admin dashboard, uploads tailored render previews, and updates the status to `'Ready for Download'`, triggering a real-time row in the `notifications` table for the user.

Examiner Psychology: Demonstrates complete business logic and role-based access control (RBAC).

Q18 What is Tone Mapping and why was ACESFilmicToneMapping used?

Model Answer: WebGL renders high-dynamic-range (HDR) light calculations that must be clamped to low-dynamic-range (LDR) monitors. Without tone mapping, highlights blowout into solid white patches. We enabled `THREE.ACESFilmicToneMapping` with an exposure of 1.1 in `viewer3d.js`, reproducing the cinematic color response curve used by modern film production and game engines for realistic fabric highlights.

Examiner Psychology: Demonstrates sophisticated understanding of photorealistic shading and rendering art.

Q19 What are the future enhancements planned for Style360?

Model Answer: 1. **Physics-based cloth simulation:** Integrating Cannon.js or Ammo.js WebAssembly physics for dynamic fabric swaying during motion. 2. **Augmented Reality (WebXR):** Projecting the 3D avatar directly into the user's living room using smartphone AR capabilities. 3. **Automated body measurement extraction:** Deriving bust, waist, and hip circumferences from frontal and profile photographs to provide exact sizing tags (S/M/L/XL).

Examiner Psychology: Shows vision, scientific ambition, and awareness of emerging technology trends.

Q20 What was your specific individual contribution to this project?

Model Answer: I was responsible for architecting the end-to-end integration: 1. Engineering the PHP 8.2 backend gateway, database schema, and self-healing connection logic. 2. Integrating the Fal.ai VTON and Tripo3D V3 APIs with automated background removal and MD5 hash caching. 3. Building the Three.js WebGL 3D graphics viewport with auto-fit Box3 camera framing, 16x anisotropic filtering, and the CORS streaming proxy. 4. Implementing the Gemini Vision AI pre-flight gender and garment validation guards.

Examiner Psychology: Deliver this answer with confidence and clarity. Ownership of key technical milestones scores maximum marks.

Q21 Is Style360 built using React? Why was Vanilla JavaScript selected over frameworks like React or Angular?

Model Answer: No, Style360 is deliberately built using modern native Vanilla JavaScript (ES6+), HTML5, and CSS3 alongside Three.js and Google's <model-viewer> Web Component. While React is well-suited for traditional corporate CRUD applications, high-performance 3D WebGL computer graphics platforms face critical architectural constraints:

- 1. Direct Canvas & GPU Memory Control:** Three.js operates an imperative 60-FPS animation render loop (`requestAnimationFrame`) requiring microsecond-level manual disposal of textures, geometries, and GPU vertex buffers. React's Virtual DOM diffing engine adds unnecessary reconciliation overhead and frequent garbage collection cycles, causing noticeable frame drops and stuttering in real-time 3D viewports.
- 2. Zero Build Overhead & Instant Compilation:** Native ES6+ requires zero Webpack, Vite, or Babel bundling steps. It loads instantly in the browser with microsecond DOM updates and zero bloated `node_modules` dependencies (which typically exceed 200MB+ in React setups).
- 3. Lightweight Reactive State Architecture:** Instead of heavy Redux or Context state managers, Style360 uses lightweight, event-driven state orchestrators (`tryonState` , `window.StudioController`) synchronized with browser `localStorage` , delivering full reactivity with zero framework footprint.

Examiner Psychology: Examiners love to ask this question to probe whether you understand framework trade-offs or merely follow hype. Explaining the GPU memory management and 60 FPS WebGL rendering loop proves high engineering competence.

A viva examination is won or lost during the live demonstration. Examiners look for confidence, smooth system flow, and resilience against unexpected input. Follow this battle-tested step-by-step presentation script.

8.1 Foolproof 5-Minute Demonstration Script

Minute 1: Introduction & Problem Context

Executive Intro

Open `index.html`. Explain the high return rates in fashion e-commerce (30-40%) caused by static photos. Point out the elegant, modern UI design, garment catalog, and category filtering (Wedding, Formal, Casual).

Minute 2: Model Upload & Intelligent Safety Guard

Pre-Flight Safety

Upload a photo. Show how the client-side pre-validator and `validate_gender.php` instantly verify the image via Gemini Vision AI. Explain how it detects full-body stature and auto-scales without cropping feet.

Minute 3: 2D Virtual Try-On Synthesis

Generative AI

Select a garment and click "⚡ Generate Try-On". Show the real-time progress indicator. When the photorealistic output appears, highlight the fabric drape, realistic shadows under the collar, and preserved body posture.

Minute 4: 3D Mesh Reconstruction & 360° Orbit View

WebGL Graphics

Click "Generate 3D Model" (or navigate to **My 3D Models** → 🔍 **View Model**). Interact with the Three.js viewport: orbit 360°, zoom in to show fabric texture sharpness (16x anisotropic filtering), and explain the auto-framing trigonometry.

Minute 5: AI Stylist & Database Verification

Wrap-up & Proof

Open the floating AI Stylist chatbot, ask "What shoes match this outfit?" and show Gemini's multimodal response. Finally, quickly open phpMyAdmin / MySQL to prove records are saved in `try_on_history`.

8.2 Viva Defense Tips (Sinhala Guidance)

💡 **Viva එක සාර්ථකව ජයගැනීමට වැදගත් උපදෙස් (Sinhala Guide):**

- **Confidence එක තියාගන්න:** Examiner ප්‍රශ්නයක් ඇහුවම බය වෙන්න එපා. Style360 කියන්නේ සැබෑ AI API 3ක් (Fal.ai, Tripo3D, Gemini) සහ Three.js 3D WebGL එකතු කරපු Advanced Architecture එකක්.
- **"Why PHP?" ඇහුවොත්:** "PHP පරණයි නේද?" කියල අහන්න පුළුවන්. ඒකට කියන්න: "PHP 8.2 එක Stateless, Fast cold-start තියෙනවා, ඒ වගේම cURL හරහා Binary GLB 3D files direct stream කරන්න Node.js

වලට වඩා *memory-efficient*" කියලා.

- **"CORS issue එක ආවෙ කොහොමද?" ඇහුවොත්:** Tripo3D එකෙන් එන 3D GLB file එක browser එකෙන් direct load කරන්න ගියාම CORS header නැති නිසා block වෙනවා. ඒක solve කරන්න අපි `backend/proxy_glb.php` ලියල streaming proxy එකක් සහ local disk cache එකක් හදලා කියන්න.
- **"Image එක crop වෙන්නෙ නැත්තෙ ඇයි?" ඇහුවොත්:** `FalService.php` එකේ අපි Custom Aspect-Ratio detection ලියල, Full-body photo එකක් ආවම Background corner color එක sample කරලා 576x1024 / 768x1024 canvas එකකට Pillarbox කරන නිසා කකුල් හෝ ඔලුව කැපෙන්නේ නෑ කියන්න.
- **"React වලින්ද හැදුවේ?" ඇහුවොත්:** "නැහැ සර්/මැඩම්, Style360 හදලා තියෙන්නේ Pure Vanilla JavaScript (ES6+), HTML5, CSS3 සහ Three.js WebGL Engine එකෙන්. React වැනි Virtual DOM diffing frameworks වල 60 FPS Three.js rendering වලදී garbage collection lags එනවා සහ bundle size එක 200MB+ වැඩි වෙනවා. Three.js එකේ direct canvas memory control, microsecond DOM update speed, සහ zero build-step performance එක ගන්න අපි Native Vanilla JS තෝරාගන්නා" කියලා විශ්වාසයෙන් පැහැදිලි කරන්න.
- **Code එක පෙන්නන්න කිව්වොත්:** Direct `backend/FalService.php` , `backend/generate_3d.php` , සහ `frontend/js/viewer3d.js` open කරලා functions පෙන්නන්න.